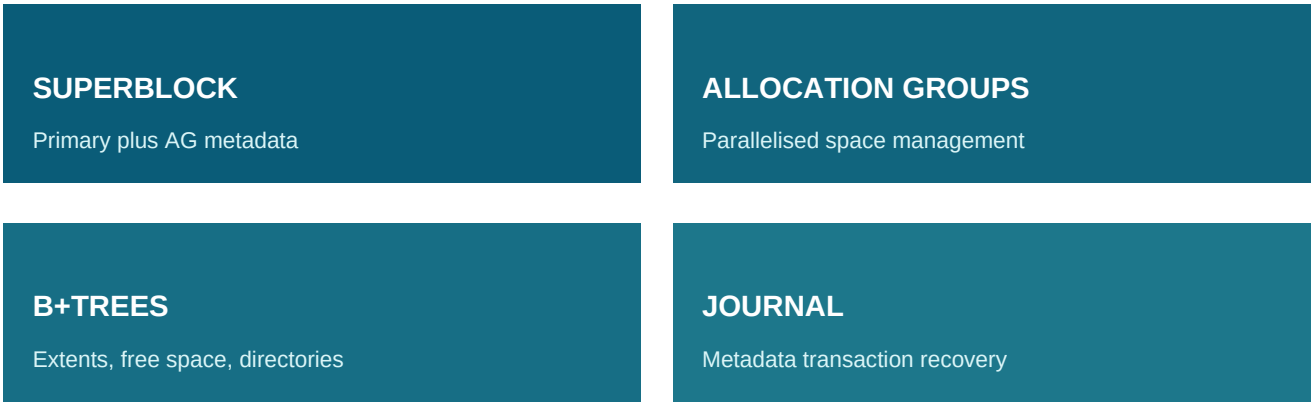
XFS - SGI high-performance filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

A journalling, extent-based, allocation-group-oriented filesystem developed for IRIX and later integrated into Linux.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

XFS - SGI high-performance filesystem

HISTORICAL CHRONOLOGY

Silicon Graphics developed XFS in the 1990s for the high-throughput storage demands of IRIX workstations and servers. It became an important filesystem in the later IRIX 5 era and was subsequently released as open source and ported to Linux, reaching mainline Linux in the 2001 period. The original motivation was scalable large files, streaming I/O and concurrent use on large disk systems.

XFS has continued to evolve in Linux through improved allocation, metadata checksums, reflinks, reverse mapping and other features. Newer on-disk versions and feature bits may change structural details. A historical IRIX-era XFS volume and a modern Linux-formatted XFS filesystem share architectural roots but not necessarily an identical set of supported metadata facilities.

XFS - SGI high-performance filesystem

ORIGINAL ON-DISK ARCHITECTURE

XFS divides the volume into allocation groups (AGs), each with its own metadata that allows parallel free-space and inode allocation. AG headers, superblocks and B+trees represent free extents and per-group inode state. Inode forks describe regular data and extended-attribute mappings, and files use extent-based allocation rather than a global FAT chain.

Directory formats vary with directory size, from compact inode-resident forms to block and B+tree arrangements. The on-disk B+tree families have separate purposes: free-space by block or length, inode indexing, extent mappings and newer reverse-mapping/reference-count trees where enabled. Parsers must interpret AG-relative versus filesystem-wide addresses correctly.

XFS - SGI high-performance filesystem

ALLOCATION AND DIRECTORIES

Allocation groups make it possible to scale concurrent writers and keep locality within large filesystems. The allocator searches free-space structures and chooses extents based on size, locality and current fragmentation. Delayed allocation and speculative preallocation can improve sequential performance. Extent maps may later change through defragmentation or reflink copy-on-write operations on formats that support them.

Directory entries map names to inodes, but the representation is not identical for all directory sizes. Extended attributes are managed in inode forks or separate blocks according to storage needs. Inode generation numbers and allocation records are important for safely interpreting stale references. Some metadata structures have physical checksum fields in modern CRC-enabled formats.

XFS - SGI high-performance filesystem

INTEGRITY AND RECOVERY

XFS uses metadata journalling to make multi-record metadata updates recoverable. The log records intents and changes required to bring the filesystem back to a consistent transaction boundary after an unexpected shutdown. It does not normally journal all user file data. Recovery should replay the log when appropriate; offline repair tools inspect wider space-management, inode and namespace consistency.

XFS metadata CRCs and owner/identity checks strengthen corruption detection on newer format versions. Repairing an in-use XFS volume independently is unsafe. Logs and deferred operations make recovery state dependent on more than the superblock, and a forceful log discard can lose recoverable metadata changes.

XFS - SGI high-performance filesystem

DESIGN RATIONALE AND LIMITATIONS

XFS favours scaling and high throughput rather than minimising bookkeeping for tiny removable volumes. Allocation groups, B+trees and extent descriptors reduce global bottlenecks but increase implementation complexity. Relocation of individual data extents is possible under correctly qualified mechanisms, yet XFS's internal allocator retains authority over future placement.

The extent of shrink, copy-on-write, reflink, quota or realtime-device capabilities depends on version and implementation. No external utility should claim general crash-safe physical placement merely because it has produced a contiguous list of sectors. XFS is fundamentally a metadata-rich Unix filesystem, not a FAT variant.

XFS - SGI high-performance filesystem

INFILTRATOROS INTEGRATION BOUNDARY

XFS.so must parse actual allocation-group and B+tree semantics and refuse unknown incompatibility features. Its capability declarations and any kernel module qualification are implementation facts, separate from SGI’s original design. An analysis-only support level must never be presented as full writable mounting.

Source documentation and Linux on-disk definitions should be consulted when validating extent maps, directory formats, CRC fields and log state. The conceptual media map is not a binary layout; every allocation group’s real offsets and geometry come from the filesystem superblock.

XFS - SGI high-performance filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. Linux kernel: The SGI XFS Filesystem

https://www.kernel.org/doc/html/latest/admin-guide/xfs.html

2. XFS project technical documentation

https://xfs.wiki.kernel.org/

3. SGI IRIX historical platform timeline

https://archive.irixnet.org/sgistuff/software/irixintro/index.html

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

XFS / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. Primary superblock and allocation group headers.
2. AGF/AGI metadata: free space and inode allocation summaries.
3. Free-space B+trees by physical address and extent length.
4. Inode forks: data extents, attribute extents or indexed mappings.
5. Directory block/leaf/node formats vary with directory size.
6. Log records and deferred intents: metadata crash recovery.
7. CRC-enabled formats: owner, UUID and metadata validation.
8. Modern versions: refcount and reverse mapping trees.

XFS / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. 1990s: SGI develops XFS for IRIX large-file workloads.

2. Later 1990s: XFS matures on SGI servers and workstations.

3. 2001 period: Linux mainline adopts open-source XFS.

4. 2010s onward: modern CRC, reflink and reverse-map features appear.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

XFS / technical detail

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Verify allocation group geometry and AG-relative addressing.

2. Check free extent trees agree with inode allocation.

3. Interpret each directory format according to enabled version flags.

4. Replay metadata log only through qualified recovery path.

5. Never treat an XFS extent relocation as a permanent allocator reserve.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

XFS / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read the XFS superblock to determine filesystem block and sector sizes, allocation-group count, inode geometry, feature flags and log layout.

2. Translate filesystem block numbers and allocation-group-relative references using the superblock geometry; different XFS address types are not interchangeable.

3. Inspect each allocation group’s headers and free-space B+trees. Allocation groups improve scalable allocation but also require per-group consistency.

4. Locate inodes through the inode allocation B+tree and the inode-number encoding. Inode records may contain local, extent or B+tree data forks.

5. Decode directory structures by their format generation and size. A small local directory can grow into block, leaf or node forms.

6. Traverse file extent lists and B+tree roots while distinguishing delayed allocation in memory from persistently allocated disk blocks.

7. Examine the internal log or separately configured log device and journal sequence before attempting metadata replay.

8. For CRC-enabled v5 filesystems, validate the metadata header identities, owner references and checksums; older v4 formats have different protection.

XFS / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Verify allocation-group boundaries, free-space records and inode allocation state; a consistent root superblock alone cannot certify every group.

2. Check log identity and cycle ordering before recovery. External log devices must belong to the same filesystem instance.

3. Validate B+tree record ordering, sibling relationships and owner references where supported; damaged traversal can otherwise create false free-space claims.

4. Never treat speculative preallocation or an unwritten extent as already initialized user content.

5. Respect XFS version and feature flags, including metadata CRC, reflink and reverse mapping support, when decoding structures.

6. Do not shrink or rewrite a volume merely by changing its nominal block count: allocation groups and on-disk geometry must be reconstructed by qualified tools.

7. Use a preserved copy for xfs_repair-style investigation. Log handling and destructive metadata rebuilds are distinct stages.

8. Historical SGI XFS design and modern Linux extensions must be separated when attributing features to a particular disk format revision.